

# Spartan

## 1. IDE

We strongly suggest use cursor (<https://cursor.com>) to help you fix or write code via the help of AI.

You should have a pro version using your university email:

<https://cursor.com/students>. Find the word: verify status.

## 2. Config ssh

Install Remote ssh extension. And config your Spartan account. You can either use passwd or use your key to avoid input passwd <https://dashboard.hpc.unimelb.edu.au/ssh/>

```
Host spartan
  HostName spartan.hpc.unimelb.edu.au
  User Your_Unimelb_Username
  PreferredAuthentications publickey
  IdentityFile ~/.ssh/id_rsa
```

## 3. Connect to spartan and terminal (you do not need VPN)

Connect to spartan via remote ssh extension (click bottom left). Then use Cursor's default terminal.

Once you connected to spartan your terminal will locate you into spartan server automatically.

You **MUST** to work/install everything under your project folder `/data/gpfs/projects/punimxxxx/yourusername/`. Install **miniconda** **Installing Miniconda** or mamba or uv etc... and Make sure your miniconda and your data **in NOT** in your home directory i.e. `/home/Your_Unimelb_Username`.

## 4. Use interactive mode

You should use interactive mode to debug your code. In this case you will automatically get gpu to run your python code by `python xxx.py`. If you want 30 minutes to debug run:

```
sinteractive -p interactive --time=00:30:00 --cpus-per-task=8 --partition=gpu-a100-short --nodes=1 --gres=gpu:1
```

Change the time depend on how long you want to debug using the gpu.

## 5. Submit a task

If you think your code is ready you can submit a task to run (enjoy your coffee or paper writing :)).

First find out which gpu you want to submit: `spartan-weather`

You will see:

```

$ spartan-weather
Spartan Weather Report (20250808 - 10:01)
-----
GPFS usage: 2020.92TB Free: 2158.46TB (48%)
Partition usage:
[ sapphire ]: Total active jobs: 1969 Total pending/queued: 3918
  9112 cores used out of 9472 (96.19%): 74 out of 74 nodes used (100.00%)
[ cascade ]: Total active jobs: 181 Total pending/queued: 3199
  2236 cores used out of 2376 (94.10%): 33 out of 33 nodes used (100.00%)
[ interactive ]: Total active jobs: 40 Total pending/queued: 0
  131 cores used out of 512 (25.58%): 3 out of 4 nodes used (75.00%)
[ bigmem ]: Total active jobs: 8 Total pending/queued: 0
  332 cores used out of 720 (46.11%): 6 out of 10 nodes used (60.00%)
[ long ]: Total active jobs: 2 Total pending/queued: 0
  33 cores used out of 66 (50.00%): 1 out of 2 nodes used (50.00%)
[ cascade,sapphire,interactive,bigmem,long ]: Total active jobs: 2200 Total pending/
  11844 cores used out of 13224 (89.56%): 117 out of 123 nodes used (95.12%)
[ gpu-a100 ]: Total active jobs: 80 Total pending/queued: 14
  537 cores used out of 864 (62.15%): 26 out of 27 nodes used (96.29%)
[ gpu-a100-short ]: Total active jobs: 4 Total pending/queued: 3
  18 cores used out of 64 (28.12%): 2 out of 2 nodes used (100.00%)
[ gpu-h100 ]: Total active jobs: 37 Total pending/queued: 51
  237 cores used out of 640 (37.03%): 10 out of 10 nodes used (100.00%)

GPU usage in the [ gpu-a100 ] partition: 103 / 108 cards in use (95.37%)
GPU usage in the [ gpu-a100-short ] partition: 7 / 8 cards in use (87.50%)
GPU usage in the [ gpu-h100 ] partition: 10 / 10 cards in use (100.00%)

Private partition usage:
[ feit-gpu-a100 ]: Total active jobs: 72 Total pending/queued: 74
  168 cores used out of 672 (25.00%): 21 out of 21 nodes used (100.00%)
[ deeplearn ]: Total active jobs: 59 Total pending/queued: 60
  556 cores used out of 992 (56.04%): 30 out of 30 nodes used (100.00%)
[ fos-gpu-l40s ]: Total active jobs: 7 Total pending/queued: 0
  194 cores used out of 768 (25.26%): 7 out of 12 nodes used (58.33%)

GPU usage in the [ feit-gpu-a100 ] partition: 84 / 84 cards in use (100.00%)
GPU usage in the [ deeplearn ] partition: 104 / 124 cards in use (83.87%)
GPU usage in the [ fos-gpu-l40s ] partition: 19 / 48 cards in use (39.58%)

```

Use this if FEIT needs queuing up

Use this as it belongs to FEIT

Then do: `vim your_task_name.slurm`

write:

```

#!/bin/bash
#SBATCH --partition=feit-gpu-a100
#SBATCH --account=punim2243
#SBATCH --qos=feit
#SBATCH --nodes=1
#SBATCH --job-name="fla-tts"
#SBATCH -o "slurm-%N.%j.out" # STDOUT
#SBATCH -e "slurm-%N.%j.err" # STDERR

```

```

#SBATCH --ntasks=1
#SBATCH --gres=gpu:1
#SBATCH --mail-user=yourusername@unimelb.edu.au
#SBATCH --mail-type=FAIL
#SBATCH --time=0-12:00:00
#SBATCH --mem=64G
# Send yourself an email when the job:
# aborts abnormally (fails)
#SBATCH --mail-type=FAIL
# begins
#SBATCH --mail-type=BEGIN
# ends successfully
#SBATCH --mail-type=END

cd /data/gpfs/projects/project_name/your_project || exit 1 # Exit if cd fails

module purge
# Load CUDA version compatible with the cuDNN module
module load CUDA/12.4.1
module load cuDNN/9.6.0.74-CUDA-12.4.1

# Activate the bash virtual environment
conda activate your_environment
# run the code
python yoursript.py

```

You MUST need to 1. cd to your \*.py folder 2. activate your env before submit slurm.

If you everything is ready. Then:

```

sbatch your_task_name.slurm

```

you can check your submitted task ID via:

```

squeue --me

```

And if you want to stop:

```

scancel task_ID

```

Enjoy!